

Cours 5 : Typage avancé

2025-2026 (OAS381-dirty)

1/42

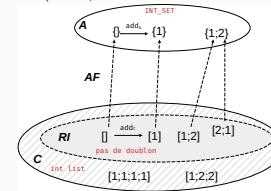
Thèmes

- types fantômes, types singletons, types uniques, *type-state*
- types (algébriques) non uniformes, généralisés
- types enregistrements, types objets (au sens P.O.O.)

2/42

Représentation par un type concret

1. Le type abstrait (A) se situe dans le monde mathématique.
2. Choix d'un type concret C et d'une fonction d'abstraction $AF : C \rightarrow A$
 - AF pas implémentable mais à documenter
 - Surjective
 - Pas forcément injective (cf. $\{1;2\}$)
 - Généralement partielle, de domaine de définition RI



3. Une opération concrète est correcte si, quand elle respecte ses éventuelles préconditions, elle commute avec AF sur RI .

6/42

Invariant de représentation

RI = invariant de représentation $\leftrightarrow$ souvent implémentable.

```
module NoDupList : INT_SET = struct
  type t = int list
  (* AF: la liste [a1; ...; an] représente l'ensemble {a1, ..., an}. *)

  (* RI: la liste ne contient pas de doublons. *)
  (* En phase de débogage : *)
  let check_rep l =
    if List.length (sort_uniq Int.compare l) = length l then l
    else failwith "RI"
  (* En phase d'exploitation : let check_rep l = l
    ou, mieux, emploi d'une version simplifiée et peu coûteuse *)

  let empty = check_rep []
  let member x s = List.mem x (check_rep s)
  let add x s = check_rep (if member x (check_rep s) then s else x :: s)
end
```

- Faut-il vérifier RI pour les arguments des fonctions d'observation (cf. deux `check_rep s` ci-dessus) ?
- Quid des fonctions privées ?

7/42

Les types dans OCAML

- le langage est fortement typé
- le système de types est très riche et expressif
- les types peuvent servir de spécification très fine
- on se rapproche des pré/post-conditions
- ces spécifications sont vérifiées par le compilateur
- Conclusion: il faut exploiter les types

le reste du monde

La plupart des constructions présentées existent ou peuvent être reproduites, plus ou moins complètement, dans d'autres langages fortement typés avec polymorphisme paramétrique (par exemple: C++, Java, Rust, F#, Haskell, ...)

3/42

Types abstraits

Types fantômes

Types fantômes

Définition

Un type *fantôme* est un type paramétré :

1. dont au moins un des paramètres (*variable fantôme*) n'apparaît pas dans la définition des *valeurs* de ce type
2. dont la définition est abstraite par une signature

Exemples

```
• module type S = sig type _ t ...
  module M : S = struct type _ t = int ...

• module type S = sig type (_,_) t ...
  module M : S = struct type ('a, 'b) t = Nil | Cons of 'b * ('a, 'b) t ...
```

Usage

- caractériser un état interne/caché (*type-state*)
 $\leadsto$ plutôt pour du code impératif (ou monade d'état, cf. cours 6)
- ne pénalise pas l'exécution (*zero cost abstraction*)

9/42

Types abstraits

"Type abstrait" : ensemble de valeurs abstraites caractérisé par des opérations et leurs spécifications [Liskov & Zilles, 1974]

- Séparation entre usage et implémentation
- Importance de la notion d'invariant

```
module type INT_SET = sig
  type t
```

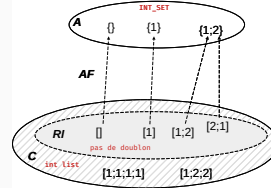
```
  val empty : t
  val member : int -> t -> bool
  val add : int -> t -> t
end
```

Ex : représentation par des listes sans doublon.

5/42

Représentation par un type concret

1. Le type abstrait (A) se situe dans le monde mathématique.
2. Choix d'un type concret C et d'une fonction d'abstraction $AF : C \rightarrow A$
 - AF pas implémentable mais à documenter
 - Surjective
 - Pas forcément injective (cf. $\{1;2\}$)
 - Généralement partielle, de domaine de définition RI



6/42

Exemple de type fantôme: spécification

- on veut imposer la lecture du premier caractère d'un fichier
- on définit l'interface `FichierLecture1Car`
- le paramètre du type `_ fichier`, prenant les valeurs `debut` et `fin`, définit l'état interne du fichier

```
module type FichierLecture1Car = sig
  type debut
  type fin
  type _ fichier
  val open : string -> debut fichier
  val read : debut fichier -> char * fin fichier
  val close : fin fichier -> unit
end
```

10/42

Exemple de type fantôme: réalisation

```
module Impl : FichierLecture1Car = struct
  type debut
  type fin
  type _ fichier = in_channel
  let open nom = open_in nom
  let read f = (input_char f, f)
  let close f = close_in f
end
```

Il est nécessaire d'imposer un usage purement séquentiel du fichier lu

- i.e. Interdire:

```
let wrong = let f = Impl.open "toto" in (Impl.read f, Impl.read f, ...)
```
- mais autoriser:

```
let lire_char nom =
  let f = Impl.open nom in
  let (c, f) = Impl.read f in
  Impl.close f;
  c
```

11/42

Types non uniformes

Types non uniformes

Définition

Un type (récursif) non uniforme 'a t fait apparaître des instances différentes du paramètre dans sa définition, **fonctions** de 'a.

Exemples

- listes alternées:

```
type ('a, 'b) alt_list = | Nil | Cons of 'a * ('b, 'a) alt_list
```

- arbres binaires équilibrés:

```
type 'a perfect_tree = | Empty | Node of 'a * ('a * 'a) perfect_tree
```

Usage

- représenter des invariants de structure "descendants"
- meilleure spécification
- nécessite la "récursion polymorphe"

13/42

Aparté : variables de type

Dans une signature ou dans l'interprète :

```
val f : 'a -> 'b -> 'a (aussi noté val f : 'a 'b . 'a -> 'b -> 'a)  
signifie: val f : ∀ 'a . ∀ 'b . 'a -> 'b -> 'a
```

Mais dans une *définition* let f : 'a -> 'b -> 'a = e, 'a -> 'b -> 'a signifie juste que l'on contraint le type de e à avoir la *forme* 'a -> 'b -> 'a, p. ex :

```
> let f : 'a -> 'b -> 'a = fun x y -> x + 1;;  
val f : int -> 'b -> int = <fun>
```

Si on souhaite quantifier les variables universellement dans une *définition*, il faut employer la notation : let f : 'a 'b . 'a -> 'b -> 'a = e.

15/42

Retour aux arbres parfaits

```
let rec depth : 'a . 'a perfect_tree -> int = function  
| Empty -> 0  
| Node (_, sub) -> 1 + depth sub;;  
val depth : 'a perfect_tree -> int = <fun>
```

16/42

Un type non-uniforme: les arbres parfaits

Fonction qui calcule la profondeur d'un arbre parfait :

```
let rec depth = function  
| Empty -> 0  
| Node (_, sub) -> 1 + depth sub;;
```

14/42

Un type non-uniforme: les arbres parfaits

Fonction qui calcule la profondeur d'un arbre parfait :

```
let rec depth = function  
| Empty -> 0  
| Node (_, sub) -> 1 + depth sub;;  
Error: This expression has type ('a * 'a) perfect_tree  
but an expression was expected of type 'a perfect_tree  
The type variable 'a occurs inside 'a * 'a
```

14/42

Types algébriques généralisés

Changement de notation !

```
type 'a tree =  
| Empty  
| Node of 'a tree * 'a * 'a tree
```

Intuitivement les *constructeurs* Empty et Node sont des fonctions de profil :

```
Empty : 'a tree  
Node : 'a tree * 'a * 'a tree -> 'a tree
```

OCaml offre une notation alternative pour les types algébriques qui reprend cette vision et explicite le type de retour de chaque constructeur :

```
type 'a tree =  
| Empty : 'a tree  
| Node : 'a tree * 'a * 'a tree -> 'a tree
```

Jusqu'ici, rien ne change par rapport à OCaml "de base"...

18/42

Un type non-uniforme: les arbres parfaits

Fonction qui calcule la profondeur d'un arbre parfait :

```
let rec depth : 'a perfect_tree -> int = function  
| Empty -> 0  
| Node (_, sub) -> 1 + depth sub;;  
Error: This expression has type ('a * 'a) perfect_tree  
but an expression was expected of type 'a perfect_tree  
The type variable 'a occurs inside 'a * 'a
```

14/42

Un type non-uniforme: les arbres parfaits

Fonction qui calcule la profondeur d'un arbre parfait :

```
let rec depth : 'a perfect_tree -> int = function  
| Empty -> 0  
| Node (_, sub) -> 1 + depth sub;;  
Error: This expression has type ('a * 'a) perfect_tree  
but an expression was expected of type 'a perfect_tree  
The type variable 'a occurs inside 'a * 'a
```

« Récursion polymorphe » : depth doit avoir un type universellement quantifié pour être appliquée sur des types différents.

14/42

Changement de notation !

```
type 'a tree =  
| Empty  
| Node of 'a tree * 'a * 'a tree
```

Intuitivement les *constructeurs* Empty et Node sont des fonctions de profil :

```
Empty : 'a tree  
Node : 'a tree * 'a * 'a tree -> 'a tree
```

OCaml offre une notation alternative pour les types algébriques qui reprend cette vision et explicite le type de retour de chaque constructeur :

```
type _ tree =  
| Empty : _ tree  
| Node : 'a tree * 'a * 'a tree -> 'a tree
```

Jusqu'ici, rien ne change par rapport à OCaml "de base"...

18/42

Types algébriques généralisés (GADT)

Définition

Les types algébriques *généralisés* (GADT¹) permettent de choisir les paramètres de type librement. Syntaxe et inférence sont distincts des autres types.

Deux nouvelles possibilités

- faire varier le type de retour des constructeurs:

```
type _ repr =  
| Int : int -> int repr  
| Add : (int -> int -> int) repr
```

- variables de type n'apparaissant pas dans le type de retour:

```
type showable = Showable : 'a * ('a -> string) -> showable
```

¹Generalized Algebraic Data Type

19/42

Types algébriques généralisés

Usage

- généralisation des types non uniformes
- types singletons (une valeur, un type)
- types uniques
- permet d'exprimer la *Run Time Type Information*
- très expressif, associé au mécanisme d'**exhaustivité du filtrage**

20/42

Types singletons

Définition

Un type singleton est un type dont chaque valeur possible est associée à exactement une instance du GADT. Cela permet de « calculer dans les types » au moyen de ces valeurs.

type nat = Zero | Succ of nat
let deux = Succ (Succ Zero);;
val deux : nat = Succ (Succ Zero)

21/42

Variation du type de retour : application

(* ajouter un element en fin de liste *)
let rec snoc : 'a -> ('a,'m) nlist -> ('a,'m succ) nlist =
 fun a -> function
 | Nil -> Cons (a, Nil)
 | Cons (x,xs) -> Cons (x, snoc a xs);;

22/42

Variation du type de retour : application

(* ajouter un element en fin de liste *)
let rec snoc : 'a -> ('a,'m) nlist -> ('a,'m succ) nlist =
 fun a -> function
 | Nil -> Cons (a, Nil)
 | Cons (x,xs) -> Cons (x, snoc a xs);;

Error: This pattern matches values of type ('a, \$0 succ) nlist
but a pattern was expected which matches values of type ('a, zero) nlist
The type constructor \$0 would escape its scope

Pas un problème de récursion polymorphe, ici, mais de typage des motifs:

- Motif Nil : ('a, zero) nlist
- Motif Cons (x, xs) : ('a, n succ) nlist pour un certain n (= \$0)

23/42

Types singletons

Définition

Un type singleton est un type dont chaque valeur possible est associée à exactement une instance du GADT. Cela permet de « calculer dans les types » au moyen de ces valeurs.

type nat = Zero | Succ of nat
let deux = Succ (Succ Zero);;
val deux : nat = Succ (Succ Zero)

type zero = Zero
type 'n succ = Succ of 'n
let deux = Succ (Succ Zero);;
val deux : zero succ succ = Succ (Succ Zero)

21/42

Types singletons

Définition

Un type singleton est un type dont chaque valeur possible est associée à exactement une instance du GADT. Cela permet de « calculer dans les types » au moyen de ces valeurs.

type nat = Zero | Succ of nat
let deux = Succ (Succ Zero);;
val deux : nat = Succ (Succ Zero)

type zero = Zero
type 'n succ = Succ of 'n
let deux = Succ (Succ Zero);;
val deux : zero succ succ = Succ (Succ Zero)

type zero = private Unused_zero
type 'n succ = private Unused_succ
type _ nat = Zero : zero nat | Succ : 'a nat -> 'a succ nat
let deux = Succ (Succ Zero);;
val deux : zero succ succ nat = Succ (Succ Zero)

21/42

Variation du type de retour : application

(* ajouter un element en fin de liste *)
let rec snoc : type m . 'a -> ('a, m) nlist -> ('a, m succ) nlist =
 fun a -> function
 | Nil -> Cons (a, Nil)
 | Cons (x,xs) -> Cons (x, snoc a xs);;
val snoc : 'a -> ('a, 'm) nlist -> ('a, 'm succ) nlist = <fun>

La notation **type m** (pas d'apostrophe !) stipule que :

- (*récursion polymorphe*) **m** est quantifié universellement ;
- (*type localement abstrait*) dans les motifs, **m** est égal aux types correspondants inférés dans les motifs ; ici :
 - motif Nil : ('a, zero) nlist : ajout de zero = m
 - motif Cons (x, xs) : ('a, \$0 succ) nlist : ajout de \$0 succ = m ;

le typeur en déduit que les motifs sont bien typés (zero = m = \$0 succ).

Notation quasi-systématique quand on fait de la récursion sur un GADT !

22/42

Arithmétique dans les types (Prolog !)

On a parfois besoin de raisonner sur plus que la simple application du successeur. Exemple : spécifier l'addition au niveau des types.

$$\frac{}{0 + n = n} \text{ADD_ZERO} \qquad \frac{n + m = p}{(n + 1) + m = (p + 1)} \text{ADD_SUCC}$$

type _ nat = Zero : zero nat | Succ : 'a nat -> 'a succ nat

type (_, _, _) add =
 | Add_zero : (zero, 'n, 'n) add
 | Add_succ : ('n, 'm, 'p) add -> ('n succ, 'm, 'p succ) add

23/42

Variation du type de retour : application

(* listes de taille n *)
type (_, _) nlist =
 | Nil : ('b, zero) nlist
 | Cons : 'a * ('a, 'n) nlist -> ('a, 'n succ) nlist

22/42

Variation du type de retour : application

(* listes de taille n *)
type (_, _) nlist =
 | Nil : ('b, zero) nlist
 | Cons : 'a * ('a, 'n) nlist -> ('a, 'n succ) nlist

(* version totale de [hd] *)
let hd : ('a, _ succ) nlist -> 'a =
 function Cons (h, _) -> h;;
val hd : ('a, 'b succ) nlist -> 'a = <fun>

22/42

Arithmétique dans les types : induction

Montrer que $\forall a. a + 0 = a$:

$$\frac{\text{ADD_ZERO} \quad \frac{p + 0 = p \vdash p + 0 = p}{p + 0 = p \vdash (p + 1) + 0 = (p + 1)} \text{HYP} \quad \text{ADD_SUCC}}{\forall a. a + 0 = a} \text{IND.}$$

24/42

Arithmétique dans les types : induction

Montrer que $\forall a. a + 0 = a$:

$$\frac{\text{ADD_ZERO} \quad \frac{p + 0 = p \vdash p + 0 = p}{p + 0 = p \vdash (p + 1) + 0 = (p + 1)} \text{HYP} \quad \text{ADD_SUCC}}{\forall a. a + 0 = a} \text{IND.}$$

let rec add_zero_right: type a. a nat -> (a, zero, a) add = fun a ->
 match a with
 | Zero -> Add_zero
 | Succ p -> Add_succ (add_zero_right p)

24/42

Arithmétique dans les types : induction

Montrer que $\forall a. a + 0 = a$:

$$\frac{\text{ADD_ZERO} \frac{}{0 + 0 = 0} \quad \frac{p + 0 = p \vdash p + 0 = p \quad \text{ADD_SUCC}}{p + 0 = p \vdash (p + 1) + 0 = (p + 1)} \text{IND.}}{\forall a. a + 0 = a}$$

let rec add_zero_right: type a. a nat -> (a, zero, a) add = fun a ->
 match a with
 | Zero -> Add_zero
 | Succ p -> Add_succ (add_zero_right p)

Rappel Coq :

Lemma add_zero_right: forall a, add a Zero a.
Proof.
 intro. induction a.
 - apply Add_zero.
 - apply Add_succ. apply IHm.
Qed.

24/42

Types uniques

Définition

Un type unique est un type associé à une seule donnée dans tout le programme. Un type unique est créé en même temps que la valeur correspondante.

Exemples

- directement avec des GADT :

```
type unique = Unique : 'a -> unique
let create_unique v = Unique v
```
- Note : $\forall a. a \rightarrow \text{unique} \cong (\exists a. a) \rightarrow \text{unique}$ ($a \notin \text{unique}$)
- avec un type abstrait et un module de première classe (cf. annexe)

```
module type Typetunique = sig
  type unique
  val value : unique
end
(* create_unique : 'a -> (module Typetunique) *)
let create_unique (type a) (v : a) =
  (module struct type unique = a let value = v end : Typetunique)
```

25/42

Type existentiel

Type de `Kp` : $\forall 'u. ((\text{pub}, 'u) \text{key} * (\text{priv}, 'u) \text{key} \rightarrow \text{key_pair}) \cong (\exists 'u. (\text{pub}, 'u) \text{key} * (\text{priv}, 'u) \text{key}) \rightarrow \text{key_pair}$

Quand `Kp` est appliqué, `'u` est connu mais il est abstrait (oublié) dans `key_pair`.

On sait juste qu'il existe une instanciation pour `'u` mais on ne peut la laisser s'échapper sans briser la barrière d'abstraction :

```
module M(R : RSA) = struct
  open R
  let x = let Kp (a,b) = create_key_pair () in 1
end;;
module M : functor (R : RSA) -> sig val x : int end
```

```
module M(R : RSA) = struct
  open R
  let x = let Kp (a,b) = create_key_pair () in a
end;;
Error: This expression has type (pub, $Kp.'u) key
      but an expression was expected of type 'a
      The type constructor $Kp.'u would escape its scope
```

28/42

Types existentiels : implémentation “bidon”

```
module Bidon : RSA = struct
  type pub
  type priv
  type ('pub_or_priv, 'u) key = unit (* int dans une vraie implémentation *)
  type 'u secret = string
  type key_pair = Kp : (pub, 'u) key * (priv, 'u) key -> key_pair
  let create_key_pair () = Kp ((), ())
  let encrypt s pubk = s
  let decrypt secret privk = secret
end
```

29/42

Types uniques : exemple

- on considère un chiffrement asymétrique de type RSA (Rivest-Shamir-Adleman)
 - clé publique pour chiffrer, clé privée pour déchiffrer
- clés publiques/privées de type `key` de même nature (entiers), mais devraient être distinguées
- la donnée chiffrée de type `secret` devrait être associée à ses clés

26/42

Types uniques : déclarations

Interface “classique” :

```
module type RSA = sig
```

```
(* clé *)
type key
(* secret produit *)
type secret
(* couple de clés publique/privée *)
type key_pair = Kp of key * key
(* creation d'une paire de clés publique/privée *)
val create_key_pair : unit -> key_pair
(* chiffrement a l'aide de la clé publique *)
val encrypt : string -> key -> secret
(* déchiffrement a l'aide de la clé privée *)
val decrypt : secret -> key -> string
end
```

27/42

Types existentiels : implémentation “bidon”

```
let () =
  let open Bidon in
  let msg = "message super important" in
  let Kp (pubk1, privk1) = create_key_pair () in
  let Kp (pubk2, privk2) = create_key_pair () in
  let secret = encrypt msg pubk1 in
  let clair = decrypt secret privk2 in (* erreur *)
  print_endline clair;;
Error: This expression has type (priv, $Kp.'u1) key
      but an expression was expected of type (priv, $Kp.'u) key
      Type $Kp.'u1 is not compatible with type $Kp.'u
```

30/42

Types existentiels : implémentation “bidon”

```
let () =
  let open Bidon in
  let msg = "message super important" in
  let Kp (pubk1, privk1) = create_key_pair () in
  let Kp (pubk2, privk2) = create_key_pair () in
  let secret = encrypt msg pubk1 in
  let clair = decrypt secret privk1 in
  print_endline clair;;
message super important
```

30/42

Types uniques : déclarations

Type fantôme (`key`) :

```
module type RSA = sig
  (* paramètres pour les clés *)
  type pub
  type priv
  (* clé publique ou privée (type fantôme) *)
  type 'pub_or_priv key
  (* secret produit *)
  type secret
  (* couple de clés publique/privée *)
  type key_pair = Kp of pub key * priv key
  (* creation d'une paire de clés publique/privée *)
  val create_key_pair : unit -> key_pair
  (* chiffrement a l'aide de la clé publique *)
  val encrypt : string -> pub key -> secret
  (* déchiffrement a l'aide de la clé privée *)
  val decrypt : secret -> priv key -> string
end
```

27/42

Types uniques : déclarations

Type fantômes (`key` et `secret`) + type existentiel :

```
module type RSA = sig
  (* paramètres pour les clés *)
  type pub
  type priv
  (* clé publique ou privée avec marqueur 'u pour l'unicité de cette clé *)
  type ('pub_or_priv, 'u) key
  (* secret produit avec la clé de marqueur 'u *)
  type 'u secret
  (* couple de clés publique/privée avec marqueur d'unicité 'u *)
  type key_pair = Kp : (pub, 'u) key * (priv, 'u) key -> key_pair
  (* creation d'une paire de clés publique/privée *)
  val create_key_pair : unit -> key_pair
  (* chiffrement a l'aide de la clé publique *)
  val encrypt : string -> (pub, 'u) key -> 'u secret
  (* déchiffrement a l'aide de la clé privée *)
  val decrypt : 'u secret -> (priv, 'u) key -> string
end
```

27/42

Conclusion

- OCAML possède un langage de types très puissant
- nombreuses modélisations possibles, purement avec des types
- abstractions à faible coût
- beaucoup d'autres aspects non évoqués
 - Objets et classes (typage structurel)
 - Variants extensibles
 - Variants polymorphes
 - Types étiquettes et options

31/42

Annexe

Exemple de type unique avec un module de première classe

Une interface `KeyPair` spécifie les modules avec un type `unique` + 2 clés.

```
module type RSA = sig
  type pub
  type priv
  type ('typ, 'uniq) key
  type 'uniq secret
  (* Tout module qui implémente cette signature a un type existentiel car,
   étant abstrait, on sait juste qu'un type concret existe qui réalise
   "unique", mais on ne peut savoir lequel. *)
  module type KEY_PAIR = sig
    type unique
    val pubk : (pub, unique) key
    val privk : (priv, unique) key
  end
  (* creation d'une paire de clés publique/privée *)
  val create_key_pair : unit -> (module KEY_PAIR)
  (* chiffrement a l'aide de la clé publique *)
  val encrypt : string -> (pub, 'uniq) key -> 'uniq secret
  (* déchiffrement a l'aide de la clé privée *)
  val decrypt : 'uniq secret -> (priv, 'uniq) key -> string
end
```

Exemple de type unique: réalisation

- une réalisation “bidon”

```
module NullCrypt : RSA = struct
  type pub
  type priv
  type ('typ, 'uniq) key = unit
  type 'uniq secret = string
  module type KEY_PAIR = sig
    type unique
    val pubk : (pub, unique) key
    val privk : (priv, unique) key
  end
  let create_key_pair () = (module
    struct
      type unique
      let pubk = ()
      let privk = ()
    end : KEY_PAIR)

  let encrypt by pk = by
  let decrypt se pk = se
end
```

Enregistrements

Types enregistrements

- type enregistrement “classique”
- champs mutables ou non
- mise-à-jour impérative (`move_x : int -> t -> unit`)
- mise-à-jour fonctionnelle (`move_y : int -> t -> t`)

```
type t = { mutable x : int; y : int }

let create x y = { x = x; y = y }

let get_y p = p.y
(* filtrage : *)
let get_y { y; _ } = y

(* mise à jour impérative *)
let move_x d v = v.x <- v.x + d

(* mise-à-jour fonctionnelle *)
let move_y d v = { x = v.x; y = v.y + d }
(* ou mieux : *)
let move_y d v = { v with y = v.y + d }
let move_y d { x; y } = { x; y = y + d }
```

Exemple de type unique: réalisation

- une application classique
- garantie sans erreurs de clé

```
let _ =
  let msg = Bytes.of_string "message super important" in
  let (module Key1) = NullCrypt.create_key_pair () in
  let (module Key2) = NullCrypt.create_key_pair () in
  let msg_secret = NullCrypt.encrypt msg Key1.pubk in
  let msg_decode = NullCrypt.decrypt msg_secret Key1.privk (*erreur si mauvaise
  msg_decode = msg
```

Types algébriques généralisés: modélisation de problèmes

- on modélise, par des types, le problème *Homme-Loup-Mouton-Chou*
- H , sur une barque à 2 places, doit transporter L , M et C , de la rive gauche à la rive droite
- M ne peut rester seul (sans H) sur une rive avec L ou C

```
(* les positions: rive gauche ou rive droite *)
type g = private G
type d = private D

(* les 4 mouvements possibles des 4 entites h, l, m, c *)
type ('h, 'l, 'm, 'c, 'hl, 'll, 'ml, 'cl) move =
| H : ('h, 'l, 'm, 'c, 'hl, 'll, 'ml, 'c) move
| HL : ('h, 'h, 'm, 'c, 'hl, 'hl, 'ml, 'c) move
| HM : ('h, 'l, 'h, 'c, 'hl, 'l, 'hl, 'c) move
| HC : ('h, 'l, 'm, 'h, 'hl, 'l, 'm, 'hl) move

(* condition de securite: m ne mange pas c et n'est pas mange par l *)
type ('h, 'l, 'm, 'c) safe =
| SafeHM : ('h, 'l, 'h, 'c) safe
| SafeHLC : ('h, 'h, 'm, 'h) safe
```

Types enregistrements *inlinés*

- type enregistrement “intégré” dans un type algébrique
- constructeurs avec arguments nommés plutôt que tuples anonymes

```
type t =
| Point of {width: int; mutable x: float; mutable y: float}
| ...

let v = Point {width = 10; x = 0.; y = 0.}

let scale l = function
| Point p -> Point {p with x = l *. p.x; y = l *. p.y}
| ...

let print = function
| Point {x; y; _} -> Format.printf "%f/%f" x y
| ....

let reset = function
| Point p -> p.x <- 0.; p.y <- 0.
| ...

let invalid = function
| Point p -> p (* incorrect *)
| ...
```

Types enregistrements polymorphes

- un type enregistrement peut être paramétrique, par exemple:

```
> type ('a, 'b) t = {x: 'a; f: 'b list -> 'b};;
type ('a, 'b) t = { x : 'a; f : 'b list -> 'b; }
> let test1 r e = r.f [e] = e;;
val test1 : ('a, 'b) t -> bool = <fun>
> let test2 r = (r.f [0] = 0) && (r.f [true] = true);;
Error: This expression has type bool but an expression was expected
of type int
```
- un type enregistrement peut contenir des champs *polymorphes*
- le champ `y` est maintenant polymorphe en `'b`:

```
> type 'a u = {x: 'a; f: 'b. 'b list -> 'b};;
type 'a u = { x : 'a; f : 'b. 'b list -> 'b; }
> let test1 r e = r.f [e] = e;;
val test1 : 'a u -> 'b -> bool = <fun>
> let test2 r = (r.f [0] = 0) && (r.f [true] = true);;
val test2 : 'a u -> bool = <fun>
```

Types algébriques généralisés: SAT-solving

- un chemin est soit terminé (tous sur la rive droite), soit un premier mouvement sûr suivi du chemin restant
- `exists_path` permet de tester l'absence de solution, par des clauses de **réfutation**, qui sont vérifiées par le système de types
- le dernier filtre est refusé, il existe bien une solution de longueur 7

```
(* les chemins, listes de mouvements surs *)
type ('h, 'l, 'm, 'c) path =
| OK : (d, d, d, d) path
| GD : (g, 'l, 'm, 'c, d, 'll, 'ml, 'cl) move * (d, 'll, 'ml, 'cl) safe
      * (d, 'll, 'ml, 'cl) path -> (g, 'l, 'm, 'c) path
| DG : (d, 'l, 'm, 'c, g, 'll, 'ml, 'cl) move * (g, 'll, 'ml, 'cl) safe
      * (g, 'll, 'ml, 'cl) path -> (d, 'l, 'm, 'c) path

let exists_path (p : (g, g, g, g) path) =
  match p with
  | OK (_, _, OK) -> .
  | GD (_, _, DG (_, _, GD (_, _, OK))) -> .
  | GD (_, _, DG (_, _, GD (_, _, DG (_, _, GD (_, _, OK)))) -> .
  | GD (_, _, DG (_, _, GD (_, _, DG (_, _, GD (_, _, DG (_, _, GD (_, _, OK)))))) -> .
```

Types algébriques généralisés: résolution de problèmes

- un chemin est soit terminé (tous sur la rive droite), soit un premier mouvement sûr suivi du chemin restant
- `exists_path` permet de tester l'absence de solution, par des clauses de **réfutation**, qui sont vérifiées par le système de types
- le dernier filtre est refusé, il existe bien une solution de longueur 7

```
(* les chemins, listes de mouvements surs *)
type ('h, 'l, 'm, 'c) path =
| OK : (d, d, d, d) path
| GD : (g, 'l, 'm, 'c, d, 'll, 'ml, 'cl) move * (d, 'll, 'ml, 'cl) safe
      * (d, 'll, 'ml, 'cl) path -> (g, 'l, 'm, 'c) path
| DG : (d, 'l, 'm, 'c, g, 'll, 'ml, 'cl) move * (g, 'll, 'ml, 'cl) safe
      * (g, 'll, 'ml, 'cl) path -> (d, 'l, 'm, 'c) path

let exists_path (p : (g, g, g, g) path) =
  match p with
  | GD (_, _, OK) -> .
  | GD (_, _, DG (_, _, GD (_, _, OK))) -> .
  | GD (_, _, DG (_, _, GD (_, _, DG (_, _, GD (_, _, OK)))) -> .
  | GD (_, _, DG (_, _, GD (_, _, DG (_, _, GD (_, _, DG (_, _, GD (_, _, OK)))))) -> .
```

Cours 6 : Monades

2025-2026 (0a5381-dirty)

1/28

Introduction

Effet : tout ce que fait une fonction au-delà d'associer un résultat à une donnée en entrée.

P. ex. étant donnée une expression `e` qui s'évalue en une valeur `v`, si remplacer `e` par `v` (dans un calcul) change le comportement du programme, c'est que `e` produit aussi des effets.

Effets disponibles en OCaml	Indisponibles
- état (y.c. d'ordre supérieur) - exceptions - entrées-sorties - non-terminaison ...	- non-déterminisme - état polymorphe - exceptions typées ...

Introduction

Pour distinguer un calcul *pur* $A \rightarrow B$ d'un calcul produisant *en plus* des effets, on donne à ce dernier un type $A \rightarrow T(B)$, où T décrit les effets à l'œuvre.

Une *monade* (Moggi 1989) = un module implémentant un effet donné sous la forme de T et d'opérations sur T respectant des lois algébriques générales.

3/28

Approche monadique: les foncteurs

Interface

```
module type FONCTEUR = sig
  type 'a t
  val map : ('a -> 'b) -> ('a t -> 'b t)
end
```

Lois

```
map id      = id
map (f o g) = (map f) o (map g)
```

4/28

Monade "option" / "maybe" / "partialité" (III)

```
let mult_even xs ys =
  let trouve l = List.find_opt (fun n -> n mod 2 = 0) 1 in
  let open Maybe in
  trouve xs >=> fun x ->
    trouve ys >=> fun y ->
      return (x * y)

let xy = mult_even [ 1;3;5;8;5;4 ] [ 9;9;6;12;3 ]

val mult_even : int list -> int list -> int option = <fun>
val xy : int option = Some 48

>=> est appelé "bind" car il associe une valeur à une variable pour un certain contexte, p. ex. dans trouve xs >=> fun x -> ...
```

9/28

Opérateurs "let"

OCaml permet de définir des opérateurs *let* spéciaux :

```
let ( let* ) e f = Maybe.(e >=> f)
(* let* x = e in e2 ==> e >=> fun x -> e2 *)
(*
   \-----/
   f          *)
```

```
let mult_even xs ys =
  let trouve l = List.find_opt (fun n -> n mod 2 = 0) 1 in
  let open Maybe in
  let* x = trouve xs in
  let* y = trouve ys in
  return (x * y)
```

10/28

Approche monadique: les monades

Interface minimale

```
module type MONADE = sig
  include FONCTEUR
  (* injecte une valeur dans un calcul avec effets *)
  val return : 'a -> 'a t
  (* "bind" : composition de calculs avec effets *)
  val (>>=) : 'a t -> ('a -> 'b t) -> 'b t
end
```

Lois

```
map f x      = x >=> (fun v -> return (f v))
x >=> return x = x
return v >=> f = f v
(f >=> g) >=> h = f >=> (fun x -> g x >=> h)
```

5/28

Monade "identité"

```
module Id : MONADE with type 'a t = 'a = struct
  type 'a t = 'a
  let map f x = f x
  let return x = x
  let (>>=) x f = f x
end
```

6/28

Map2

```
module type FONCTEUR = sig
  ...
  val map2 : ('a -> 'b -> 'c) -> 'a t -> 'b t -> 'c t
end

...

module Maybe : MAYBE = struct
  ...
  let map2 f x y = match x, y with
    | None, _ | _, None -> None
    | Some x', Some y' -> Some (f x' y')
end

let mult_even xs ys =
  let trouve l = List.find_opt (fun n -> n mod 2 = 0) 1 in
  let open Maybe in
  map2 ( * ) (trouve xs) (trouve ys)
```

11/28

Monade d'environnement / "reader" (I)

```
type expr = Const of int | Var of string | Add of expr * expr

let rec eval e env = match e with
| Const c -> c
| Var v ->
  (* on ignore l'exception [Not_found] de [List.assoc] pour cet exemple *)
  List.assoc v env
| Add (e1, e2) ->
  let v1 = eval e1 env in
  let v2 = eval e2 env in
  v1 + v2

let env1 = [ ("x", 13); ("t", 1); ("y", 14) ]

let e = Add (Const 1, Add (Const 2, Var "x"))

let _ = eval e env1
```

12/28

Monade "option" / "maybe" / "partialité" (I)

```
let mult_even xs ys =
  (* renvoie le premier entier pair (s'il y en a un) *)
  let trouve l = List.find_opt (fun n -> n mod 2 = 0) 1 in
  match trouve xs with
  | None -> None
  | Some x ->
    match trouve ys with
    | None -> None
    | Some y -> Some (x * y)

let xy = mult_even [ 1;3;5;8;5;4 ] [ 9;9;6;12;3 ]

val mult_even : int list -> int list -> int option = <fun>
val xy : int option = Some 48
```

7/28

Monade "option" / "maybe" / "partialité" (II)

```
module type MAYBE = MONADE with type 'a t = 'a option

module Maybe : MAYBE = struct
  type 'a t = 'a option
  let map f x = match x with
  | None -> None
  | Some v -> Some (f v)
  let return x = Some x
  let (>>=) x f = match x with
  | None -> None
  | Some v -> f v
end
```

8/28

Monade d'environnement / "reader" (II)

```
module type READER = sig
  include MONADE
  type env

  (* chaque monade particulière a en plus ses fonctions propres *)

  (* récupère l'environnement *)
  val ask : env t

  (* modifie l'environnement localement pour un calcul *)
  val local : (env -> env) -> 'a t -> 'a t
  (* exécute un calcul pour un environnement donné *)
  val run : 'a t -> env -> 'a
end
```

13/28

Monade d'environnement / "reader" (III)

```
module Make_reader(Env : sig type t end): READER with type env = Env.t = struct
  type env = Env.t
  type 'a t = env -> 'a

  let return (v : 'a) : 'a t = fun _ -> v
  let ( >>= ) (x : 'a t) (f : 'a -> 'b t) : 'b t = fun env -> f (x env) env
  let map (f : 'a -> 'b) (x : 'a t) : 'b t = fun env -> f (x env)
  let map2 f (x : 'a t) (y : 'b t) : 'c t = fun env -> f (x env) (y env)

  let ask : env t = fun env -> env
  let local (ext : env -> env) (x : 'a t) : 'a t = fun env -> x (ext env)
  let run (x : 'a t) (env : env) : 'a = x env
end
```

14/28

Monade d'environnement / "reader" (IV)

```
module Reader = Make_reader(struct type t = (string * int) list end)
open Reader

let rec eval = function
| Const c -> return c
| Var v ->
  ask >= fun env ->
    return (List.assoc v env)
| Add (a, b) ->
  eval a >= fun a' ->
  eval b >= fun b' ->
  return (a' + b')

let _ = Reader.run (eval e) env1
```

15/28

Monade d'environnement / "reader" (V)

```
let rec eval = function
| Const c -> return c
| Var v -> map (List.assoc v) ask
| Add (a, b) -> map2 ( + ) (eval a) (eval b)

let _ = Reader.run (eval e) env1

let _ =
  Reader.run
    (Reader.local (List.cons ("z", 15))
      (eval (Add (Var "x", Add (Var "y", Var "z"))))) env1
```

16/28

Insertions

```
module type NON_DETERMINISTIC = MONADE_PLUS with type 'a t = 'a list

module List_monad : NON_DETERMINISTIC = struct
  ...
  let zero = []
  let (++) = (0)
end

let rec insertions e l = match l with
| [] -> return [e]
| t::q ->
  return (e::l)
  ++
  map (List.cons t) (insertions e q)

let rec permutations = function
| [] -> return []
| e::q -> permutations q >= insertions e
```

21/28

Une monade ensembliste ? (I)

Attention, cette monade n'empêche pas la présence de doublons :

```
> permutations [ 1;2;2;3 ];;
- : int list list =
[[1; 2; 2; 3]; [2; 1; 2; 3]; [2; 2; 1; 3]; [2; 2; 3; 1]; [1; 2; 2; 3]; ...]

(* ou encore *)

> let rec insertions e l = match l with
| [] -> return [e]
| t::q ->
  return (e::l) ++ return (e::l) (* !!! *)
  ++
  map (List.cons t) (insertions e q)
> permutations [ 1;2;2;3 ];;
- : int list list =
[[1; 2; 3]; [1; 2; 3]; ...]
```

22/28

Permutations

- le type 'a list list rend le programme peu clair
- abstraction des ens. de permutations difficile
- la structuration monadique est une solution

```
(* insertions : 'a -> 'a list -> 'a list list *)
let rec insertions e l =
  match l with
  | [] -> [ [e] ] (* insertion de e en fin *)
  | t::q -> (e::l) (* insertion de e avant t *)

  List.map (fun l -> t::l) (* on ajoute t en tete des ... *)
    (insertions e q) (* insertions de e apres t, i.e. dans q *)

(* permutations : 'a list -> 'a list list *)
let rec permutations l =
  match l with
  | [] -> [ [] ]
  | e::q -> List.(flatten (map (fun p -> insertions e p) (permutations q)))
```

17/28

Monade non-déterministe / "ensembliste" (I)

```
module type NON_DETERMINISTIC = MONADE with type 'a t = 'a list

module List_monad : NON_DETERMINISTIC = struct
  type 'a t = 'a list
  let map = List.map
  let map2 = List.map2
  let return x = [x]
  let rec (++) l f = match l with
    | [] -> []
    | t::q -> f t @ (q >= f)
  (* ou encore : *)
  let (++) x f = List.flatten (List.map f x)
end

• quelle différence avec la monade "option" ?
• applicable aux permutations ? oui mais... cf. inf.
```

18/28

Une monade ensembliste ? (II)

Et si on implémente la monade avec des listes sans doublons ?

```
module NDet : NON_DETERMINISTIC = struct
  type 'a t = 'a list
  let add x l = if List.mem x l then l else x::l
  let (++) l1 l2 = List.fold_right add l1 l2
  let return x = [x]
  let zero = []
  let map f l = List.fold_right (fun x -> add (f x)) l []
  let map2 f xs ys = List.fold_right2 (fun x y -> add (f x y)) xs ys []
  let rec (++) x f = match x with
    | [] -> []
    | t::q -> f t ++ (q >= f)
end

Pas une monade ! Pourquoi ?
```

23/28

Application à la recherche de solutions

- on cherche un élément d'un type 'a qui satisfait une propriété
ok : 'a -> bool
- on procède de proche en proche, en visitant différentes positions
- en utilisant une fonction neighbors : 'a -> 'a list qui donne les voisins d'un élément donné
- applications: rendre la monnaie, le compte est bon, labyrinthe, ...

```
(* search_list : ('a -> 'b list) -> 'a list -> 'b list *)
let rec search_list explore positions = match positions with
| [] -> [] (* ECHEC ! *)
| pos::queue -> match explore pos with
  | [] -> search_list explore queue
  | sol -> sol (* OK ! *)

(* search : ('a -> 'a list) -> ('a -> bool) -> 'a -> 'a list *)
let rec search neighbors ok pos =
  if ok pos then [ pos ] (* OK ! *) else
  match search_list (search neighbors ok) (neighbors pos) with
  | [] -> [] (* ECHEC ! *)
  | sol -> pos :: sol
```

24/28

Monade non-déterministe / "ensembliste" (II)

Ici, un intérêt de l'approche monadique est de permettre de raisonner sur une seule permutation à la fois (la généralisation "non-déterministe" étant produite par >=).

```
let rec permutations = function
| [] -> return []
| e::q -> permutations q >= fun p -> insertions e p
```

Compréhension

Dans le cas de cette monade, `permutations q >= fun p -> insertions e p` peut être vu intuitivement comme la définition d'une liste par "compréhension": "[insertions e p | p <- permutations q]".

PS : on ne peut toujours pas définir `insertions`.

19/28

Monades additives

Interface

```
module type MONADE_PLUS = sig
  include MONADE
  val zero : 'a t
  val (++) : 'a t -> 'a t -> 'a t
end
```

Lois (variables selon les publications)

zero ++ a	=	a
a ++ zero	=	a
(a ++ b) ++ c	=	a ++ (b ++ c)
zero >= f	=	zero
f >= fun _ -> zero	=	zero
(a ++ b) >= f	=	(a >= f) ++ (b >= f)

20/28

Application de l'application: rendre la monnaie

- le type `pos` représente l'état courant du problème: la monnaie à rendre et l'ensemble des pièces disponibles dans la caisse
- le critère (ok) est que le montant à rendre soit nul
- les actions possibles au voisin d'une position donnée sont: soit rendre la première pièce disponible; soit "jeter" cette première pièce

```
type pos = int * int list (* monnaie à rendre, pièces dans la caisse *)
let ok (a_rendre, caisse : pos) = a_rendre = 0
let neighbors (a_rendre, caisse : pos) = match caisse with
| [] -> []
| p::q -> (if p <= a_rendre then [(a_rendre-p, q)] else []) @ [(a_rendre, q)]
let rendre_monnaie a_rendre caisse = (* pour affichage lisible du résultat *)
let rec loop = function
| [] -> failwith "impossible"
| [ _, _ ] -> []
| (n1, _)::((n2, _)::_) as q ->
  let ps = loop q in
  let p = n1 - n2 in
  if p > 0 then p :: ps else ps
in loop (search neighbors ok (a_rendre, caisse));
rendre_monnaie 6 [ 1;1;2;1;2;2 ];;
- : int list = [1; 1; 2; 2]
```

25/28

Application à la recherche de solutions

- utiliser le type 'a NDet.t à la place de 'a list
search_list : ('a -> 'b NDet.t) -> 'a NDet.t -> 'b NDet.t
search : ('a -> 'a NDet.t) -> ('a -> bool) -> 'a -> 'a list NDet.t
- utiliser zero et ++:
let rec search_list explore positions = match positions with
| [] -> zero
| pos::queue -> explore pos ++ search_list explore queue
- reconnaitre bind:
let search_list explore positions = positions >= explore
- le traitement du résultat de search_list est également un bind:
let rec search_neighbors ok pos =
 if ok pos then return [pos]
 else neighbors pos >= fun chemin ->
 return (pos::chemin)

contrairement à la version sans monade, avec `NDet` (avec listes sans doublons), cette version renvoie toutes les solutions possibles !

26/28

Monades et encapsulation

- une monade **M** sert à encapsuler/représenter un "effet", par dessus un calcul normal
- le type '**a M.t**' est une valeur de type '**a**', obtenue au moyen de l'effet **M.t**
- le type '**a M.t**' est le plus souvent abstrait, donc l'usage de la monade **M** est diffusif
- pour isoler l'usage de **M**, on définit une fonction
`run : 'a M.t -> 'a result`, où '**a result**' dépend de '**a**', mais pas de **M.t**
- la composition de monade, i.e. d'effets, est complexe, il faut *a minima* construire une bijection '**a M1.t M2.t** <-> 'a M2.t M1.t

27/28

Des monades pour tout représenter

- variable d'état (modifiable), entrées-sorties, exceptions, non-déterminisme, calcul probabiliste, environnement, journalisation, transactions, continuations, ...
- utiles même dans un langage où les effets sont nativement présents comme OCAML
- utiles également en logique (double négation), etc
- d'autres monades seront couvertes en TD et TP

28/28

Introduction de continuation: principe

- supposons **f** : **a** -> **b**, où l'on souhaite introduire une continuation
- f** peut être appelée par **g** : **b** -> **c** quelconque
- on transforme **f** en **kf** : **a** -> (**b** -> **c**) -> **c**, intégrant ce futur
- le type **b** est devenu (**b** -> **c**) -> **c**, en fait très proche
- lien étroit avec la logique, où A est proche de $\neg\neg A = (A \rightarrow \perp) \rightarrow \perp$
- "inversion de contrôle" / principe d'Hollywood : "don't call us, we'll call you!"

5/31

Introduction de continuation: exemples

Transformation systématique appelée *Continuation-Passing Style* (CPS)

```
> let f x = 2 * x;;
val f : int -> int = <fun>
> let g x = x + 5;;
val g : int -> int = <fun>
> let kf x k = k (2 * x);;
val kf : int -> (int -> 'a) -> 'a = <fun>
> let kg x k = k (x + 5);;
val kg : int -> (int -> 'a) -> 'a = <fun>

> let h x = g (f x);;
val h : int -> int = <fun>
(* expliciter les calculs intermédiaires *)
> let h x =
  let fx = f x in
  let gfx = g fx in
  gfx;;
val h : int -> int = <fun>
> h 6;;
- : int = 17
```

6/31

Cours 7 : continuations & effets

2025-12-02 (0a5381-dirty)

1/31

Thèmes

- la notion de continuation, applicable à (presque) tous les langages
- la transformation CPS
- les effets et continuations natives
- usages: inversion de contrôle, coroutines, processus utilisateurs, compilation, etc

2/31

Introduction de continuation: exemples

Transformation systématique appelée *Continuation-Passing Style* (CPS)

```
> let f x = 2 * x;;
val f : int -> int = <fun>
> let g x = x + 5;;
val g : int -> int = <fun>
> let kf x k = k (2 * x);;
val kf : int -> (int -> 'a) -> 'a = <fun>
> let kg x k = k (x + 5);;
val kg : int -> (int -> 'a) -> 'a = <fun>

> let h x = g (f x);;
val h : int -> int = <fun>
(* expliciter les calculs intermédiaires puis introduire les continuations *)
> let h x =
  let fx = f x in
  let gfx = g fx in
  gfx;;
val h : int -> int = <fun>
> h 6;;
- : int = 17

> let kh x k =
  kf x (fun fx ->
    kg fx (fun gfx ->
      k gfx));;
val kh : int -> (int -> int) -> int = <fun>
> kh 6 (fun x -> x);;
- : int = 17
```

6/31

Introduction de continuation: exemples

```
> let rec fact n =
  if n <= 0 then 1
  else n * fact (n - 1);;
val fact : int -> int = <fun>
> let rec kfact n k =
  if n <= 0 then k 1
  else
    kfact (n - 1) (fun fact_n_1 ->
      k (n * fact_n_1));;
val kfact : int -> (int -> 'a) -> 'a = <fun>
> let fact' n = kfact n (fun fact_n -> fact_n);;
val fact' : int -> int = <fun>
```

7/31

La notion de continuation

- permet de représenter l'état d'un calcul en cours dans son contexte d'exécution, i.e. ce qui *continue l'exécution*
- supposons qu'on soit en train d'exécuter **fact** 3 (déf. usuelle de **fact**)
- à un moment de l'exécution, on en est à 3 * (2 * (**fact** 1))
- du point de vue du calcul courant (**fact** 1), la *continuation* (*courante*) peut être vue comme une fonction **fun fact_1 -> 3 * 2 * fact_1**
- attention : une continuation est relative à un contexte d'exécution**
- p. ex. dans le programme 5 + **fact** 2, la continuation de **fact** 1 est **fun fact_1 -> 5 + 2 * fact_1**

3/31

Continuation-passing style

4/31

Introduction de continuation: exemples

```
> let rec fact n =
  if n <= 0 then 1
  else n * fact (n - 1);;
val fact : int -> int = <fun>
> let rec kfact n k =
  if n <= 0 then k 1
  else
    kfact (n - 1) (fun fact_n_1 ->
      k (n * fact_n_1));;
val kfact : int -> (int -> 'a) -> 'a = <fun>
> let fact' n = kfact n (fun fact_n -> fact_n);;
val fact' : int -> int = <fun>

fact' 3
→ kfact 3 (fun fact_n -> fact_n)
      k3 = id
→ kfact 2 (fun fn2 -> k3 (3 * fn2))
      k2
→ kfact 1 (fun fn1 -> k2 (2 * fn1))
      k1
→ kfact 0 (fun fn0 -> k1 (1 * fn0))
      k0
→ k0 1
→ k1 (1 * 1)
→ k2 (2 * 1)
→ k3 (3 * 2)
→ 6
```

7/31

Introduction de continuation: exemples

```
> let rec fibo n =
  if n <= 1 then 1
  else fibo (n-1) + fibo (n-2);;
val fibo : int -> int = <fun>
> let rec kfibo n k =
  if n <= 1 then k 1
  else
    kfibo (n-1) (fun fib_n_1 ->
      kfibo (n-2) (fun fib_n_2 ->
        k (fib_n_1 + fib_n_2)));;
val kfibo : int -> (int -> 'a) -> 'a = <fun>
> let fibo' n = kfibo n (fun fib_n -> fib_n);;
val fibo' : int -> int = <fun>
```

8/31

Transformation CPS

- récursivité terminale (– de pile vs. + de mémoire)
- mieux contrôler/changer/linéariser le flot de contrôle d'un programme
- utile par exemple pour écrire un compilateur (gestion du flot de contrôle)

9/31

9/31

Continuations pour étendre un langage

Exemple: la sémantique des exceptions

- on a un langage composé de : valeurs (*val*), fonctions (*(fun v -> expr)*) et appel de fonction (*(fn ar)*)
- on souhaite implanter/représenter la sémantique des exceptions
- une exécution peut terminer (succès) ou lever une exception (échec)
- on ajoute 2 continuations : *ko*, d'échec; *ok*, de succès
- pour chacune des constructions du langage de départ
- auxquelles on ajoute le traitement des exceptions

```

[try expr1 with Exc v -> expr2] ko ok  $\triangleq$  [expr1] (fun v -> [expr2] ko ok) ok
[raise (Exc v)] ko ok  $\triangleq$  ko v
[(fun v -> expr)] ko ok  $\triangleq$  (fun v -> [expr] ko ok)
[(fn ar)] ko ok  $\triangleq$  [ar] ko (fun va -> [fn] ko (fun vf -> vf va))
[val] ko ok  $\triangleq$  ok val
    
```

10/31

10/31

10/3

Les effets : le retour de la vengeance

Rappel : un *calcul* renvoie une *valeur* et produit éventuellement des *effets*. OCaml (partie impérative) comprend des effets *prédéfinis* (entrées-sorties, exceptions, références...).

Dans un langage fonctionnel "pur", nous connaissons deux manières d'implémenter des effets :

- monades
- CPS

Ces deux manières sont dites en *style indirect*, car elles imposent "d'envelopper" le code (*via return/bind* ou passage de continuations supplémentaires) pour pouvoir être utilisées, et elles diffusent dans le reste du programme.

OCaml 5.0 (2022) a introduit un mécanisme d'effets en style direct (indiscernable d'un programme normal) ; OCaml 5.3 (2025) a ajouté une syntaxe "conviviale" pour manipuler ces effets.

13/31

13/31

```

Example : des exceptions standard aux exceptions repreneables (I)

exception Zero
exception Bad_string of string
(* ou *)
type exn += Zero : exn | Bad_string : string -> exn

let zero () = raise Zero
let bad_string s = raise (Bad_string s)

let convert s = match int_of_string s with
| 0 -> zero ()
| v -> v
| exception Failure _ -> bad_string s

let prod xs =
  try
    let xs' = List.(fold_left ( * ) 1 (map convert xs)) in
    Format.printf "Product = %d0." xs'
  with
  | Zero -> Format.printf "Null product%."
  | Bad_string s -> Format.printf "Bad string: '%s0.'" s

```

14/31

Effets & continuations natives

Continuations natives

- la transformation CPS est fastidieuse et invasive (changement du code source)
- les continuations natives existent dans de nombreux langages : `call/cc` en Scheme
- ou bien des primitives inspirées de celles-ci : `setjmp/longjmp` en C, `yield` en Python, Scala, Racket, Haskell, etc
- ces primitives permettent de faire comme si on avait ajouté des paramètres supplémentaires de continuation dans le code des fonctions
- on étudiera des continuations natives

12/31

- 12/3

```
Exemple : des exceptions standard aux exceptions reprenables (II)
```

```
> prod [ "3"; "foo"; "7"; "bar"; "2" ];;
Bad string: 'foo'
val - : unit = ()

> prod [ "3"; "foo"; "0"; "7"; "bar"; "2" ];;
Bad string: 'foo'
val - : unit = ()

> prod [ "3"; "0"; "foo"; "7"; "bar"; "2" ];;
Null product
val - : unit = ()
```

15/31

15/31

```

Example : des exceptions standard aux exceptions repreneables (III)

exception Zero

type _ eff += Bad_string : string -> int eff

let zero () = (* même code *)
let bad_string s = Effect.perform (Bad_string s)

let convert s = (* même code *)

let prod xs =
  try
    let xs' = List.(fold_left ( * ) 1 (map convert xs)) in
    Format.printf "Product = %d0." xs'
  with
  | Zero -> Format.printf "Null product@."
  | effect (Bad_string s), k ->
    Format.printf "Bad string: '%s', replaced by 10." s;
    Effect.Deep.continue k 1

```

16/31

Les effets : le retour de la vengeance

Rappel : un *calcul* renvoie une *valeur* et produit éventuellement des *effets*.
OCaml (partie impérative) comprend des effets *prédéfinis* (entrées-sorties, exceptions, références...).

12/31

13/31

Les effets : le retour de la vengeance

Rappel : un *calcul* renvoie une *valeur* et produit éventuellement des *effets*. OCaml (partie impérative) comprend des effets *prédéfinis* (entrées-sorties, exceptions, références...).

Dans un langage fonctionnel “pur”, nous connaissons deux manières d’implémenter des effets :

- monades
- CPS

Ces deux manières sont dites en *style indirect*, car elles imposent “d’envelopper” le code (*via return/bind* ou passage de continuations supplémentaires) pour pouvoir être utilisées, et elles diffusent dans le reste du programme.

13/31

13/3

```
> prod [ "3"; "foo"; "7"; "bar"; "2" ];;
Bad string: 'foo', replaced by 1
Bad string: 'bar', replaced by 1
Product = 42
val - : unit = ()
> prod [ "3"; "foo"; "0"; "7"; "bar"; "2" ];;
Bad string: 'foo', replaced by 1
Null product
val - : unit = ()
> prod [ "3"; "0"; "foo"; "7"; "bar"; "2" ];;
Null product
val - : unit = ()
```

Note : le choix de remplacer une chaîne erronée par 1 est fait hors de *convert*, par *prod* qui installe une *gestionnaire d'effet* (et d'exception) .

17/31

Effets dans OCaml 5.3 (I)

18/31

Effets dans OCaml 5.3 (II) Cette continuation est <i>linéaire</i> (<i>one-shot</i>) : elle <i>doit</i> être utilisée <i>exactement une fois</i> par le traitement : - soit avec <code>Effect.Deep.continue</code> : ('p, 'c) continuation -> 'p -> 'c, - soit avec <code>Effect.Deep.discontinue</code> qui la détruit et lève une exception. L'appel à <code>continue k x</code> fait deux choses : - reprend (<i>resume</i>) le calcul avec <code>x</code> en lieu et place du <code>perform ...</code> - réinstalle le même gestionnaire d'effets pour capturer de possibles futures levées du même effet (= effets "profonds"). Ici, <code>continue k 1</code> vient mettre <code>1</code> à la place du <code>bad_string s</code> et réinstalle le gestionnaire (d'où la seconde capture, de <code>bad_string "bar"</code>). (Des effets "superficiels" sont présents dans <code>Effect.Shallow</code>, mais ils ne bénéficient pas de la syntaxe conviviale du <code>match / with effect</code>.) 19/31	Effets dans OCaml 5.3 (III) Les effets ne sont (pour le moment) pas tracés par le système de types, d'où des erreurs dynamiques possibles : <pre>exception Unhandled : 'a t -> exn</pre> <i>Unhandled e is raised when effect e is performed and there is no handler for it.</i> <pre>exception Continuation_already_resumed</pre> <i>Exception raised when a continuation is continued or discontinued more than once.</i> 20/31	Un effet d'environnement (II) <pre>module Reader (S : sig type env end) = struct type env = S.env type _ eff += Ask : env eff let ask () = Effect.perform Ask let run f env = match f () with r -> r effect Ask, k -> Effect.Deep.continue k env end</pre> Error: The value env has type 'a but an expression was expected of type %eff This instance of %eff is ambiguous: it would escape the scope of its equation 25/31	Un effet d'environnement (III) <pre>module Reader (S : sig type env end) = struct type env = S.env type _ eff += Ask : env eff let ask () = Effect.perform Ask let run f (env : env) = match f () with r -> r effect Ask, k -> Effect.Deep.continue k env end</pre> 26/31
Varier et composer les effets (I) <pre>let p a b = print_endline ("a = " ^ a); print_endline ("b = " ^ b); String.length a + String.length b > p "hello" "world";; a = hello b = world - : int = 10 type _ eff += Print : string -> unit eff let print_endline s = Effect.perform (Print s) let p a b = (* même code *)</pre> 21/31	Varier et composer les effets (II) <pre>let hstandard f = match f () with r -> r effect Print s, k -> Stdlib.print_endline s; Effect.Deep.continue k () (* ou, car on ne fait que renvoyer la valeur calculée dans le cas nominal : *) let hstandard f = try f () with effect Print s, k -> Stdlib.print_endline s; Effect.Deep.continue k () > hstandard (fun () -> p "hello" "world");; a = hello b = world - : int = 10 let hsilent f = try f () with effect Print _, k -> Effect.Deep.continue k () > hsilent (fun () -> p "hello" "world");; - : int = 10</pre> 22/31	Un effet d'environnement (IV) Style direct ! <pre>module R = Reader(struct type env = (string * int) list end) open R let rec eval : expr -> int = function Const c -> c Var v -> let env = ask () in List.assoc v env Add (a, b) -> let a' = eval a in let b' = eval b in a' + b' let env1 = [("x", 13); ("e", 1); ("y", 14)] let e = Add (Const 1, Add (Const 2, Var "x")) > run (fun () -> eval e) env1;; val - : int = 16</pre> 27/31	Continuations <i>linéaires</i> (I) <pre>type _ eff += Decide : bool eff let decide () = Effect.perform Decide let choose_diff () = let choose (x, y) = let b = decide () in if b then x else y in let x1 = choose (15, 30) in let x2 = choose (5, 10) in x1 - x2 let pick_true f = try f () with effect Decide, k -> Effect.Deep.continue k true > pick_true choose_diff;; val - : int = 10</pre> 28/31
Varier et composer les effets (III) <pre>let hreverse f = try f () with effect Print s, k -> let v = Effect.Deep.continue k () in print_endline s; v > hreverse (fun () -> p "hello" "world");; Exception: Stdlib.Effect.Unhandled(Print("b = world")) > hstandard (fun () -> hreverse (fun () -> p "hello" "world"));; b = world a = hello - : int = 10</pre> 23/31	Un effet d'environnement (I) Rappel : passer un environnement au moyen d'une monade. <pre>type expr = Const of int Var of string Add of expr * expr ... let rec eval : expr -> int t = function Const c -> return c Var v -> ask >>= fun env -> return (List.assoc v env) Add (a, b) -> eval a >>= fun a' -> eval b >>= fun b' -> return (a' + b')</pre> Style "indirect" : >>=, return, int t 24/31	Continuations <i>linéaires</i> (II) <pre>let pick_all f = try [f ()] with effect Decide, k -> let xt = Effect.Deep.continue k true in let xf = Effect.Deep.continue k false in xt @ xf > pick_all choose_diff;; Exception: Stdlib.Effect.Continuation_already_resumed.</pre> 29/31	Continuations <i>linéaires</i> (III) Solution en utilisant le paquet <code>multicont</code> pour des continuations <i>multishot</i> : <pre>let pick_all f = let module MD = Multicont.Deep in try [f ()] with effect Decide, k -> let xt = Effect.Deep.continue (MD.clone_continuation k) true in let xf = Effect.Deep.continue k false in xt @ xf > pick_all choose_diff;; val _28 : int list = [10; 5; 25; 20]</pre> Utilisation de <code>multicont</code> pas sûre en l'état (risques liés aux optimisations effectuées par le compilateur et à l'ordre des effets). 30/31

Conclusion

Transformation CPS :

- Récursivité terminale
- Permet d'implémenter des effets et des langages en général

Système d'effets :

- Syntaxe "naturelle" pour implémenter des effets
- Limitation aux continuations linéaires
- Les effets, et plus encore leur typage, demeurent un sujet d'étude pour les concepteurs de langages de programmation